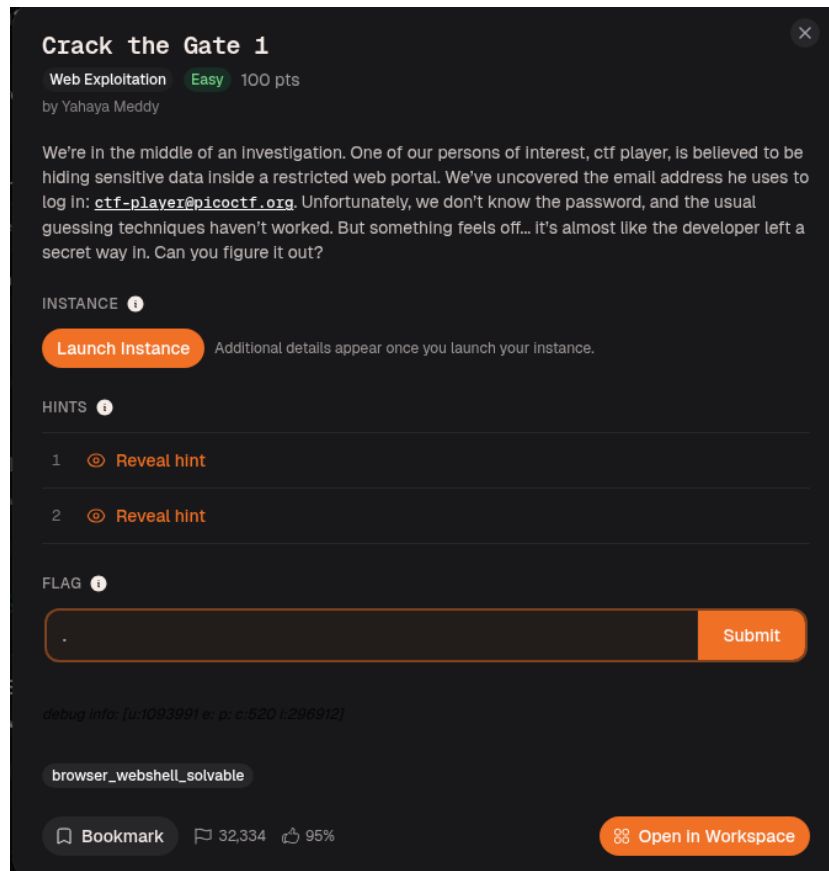


Crack the Gate 1

picoCTF - Web Exploitation - Easy (100 pts)



Challenge description: the password is unknown, normal guessing is not working, and the wording hints that a developer left a secret way in.

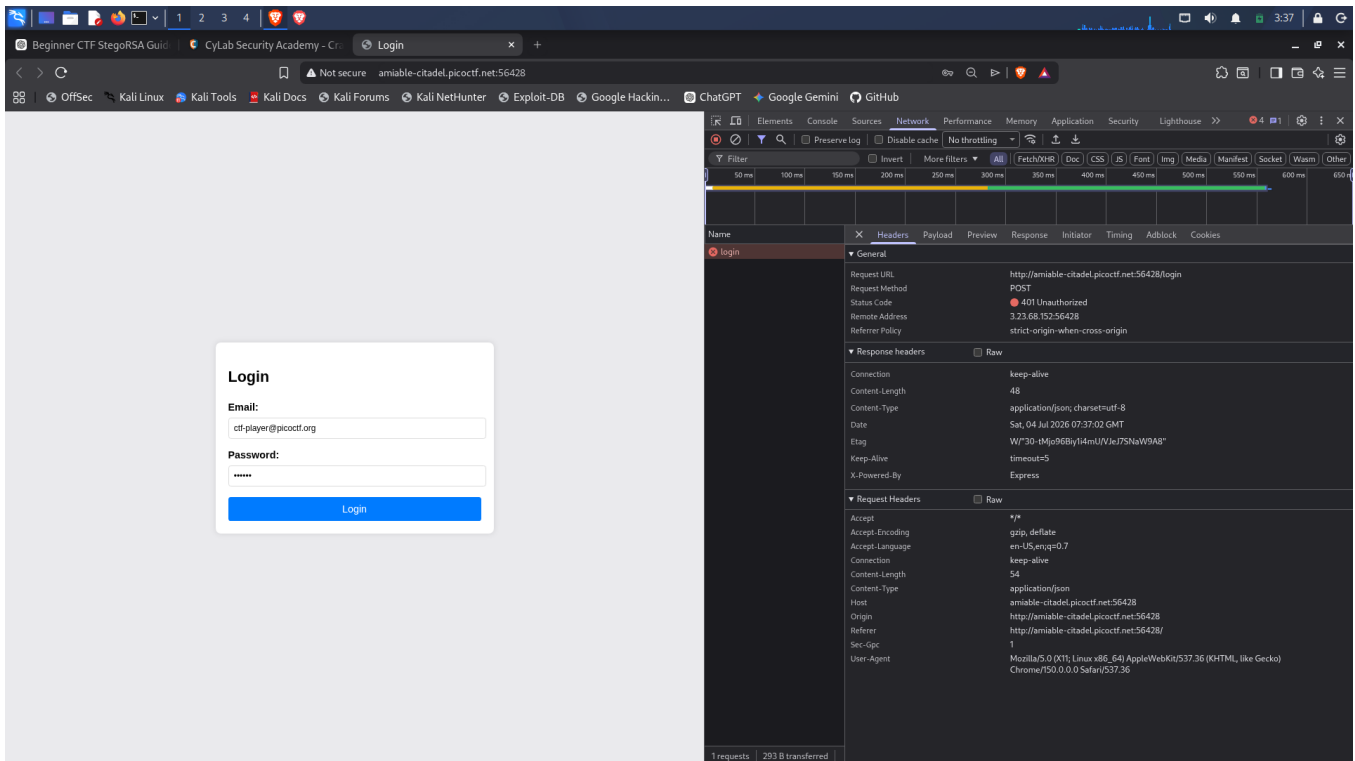
Goal and Initial Idea

The challenge gives the known email **ctf-player@picoctf.org**. I first tried a simple password, **player**, but the page returned **Invalid Credentials**. At that point I stopped guessing because the challenge wording strongly suggested a developer backdoor rather than a password brute-force problem.

Key observation: in web challenges, the visible login form is only the frontend. The real authentication happens through an HTTP request to the server, so I inspected that request in DevTools.

1. Inspecting the Login Request

I opened DevTools, went to the **Network** tab, kept the log visible, and submitted the login form again. The browser sent a POST request to `/login`.



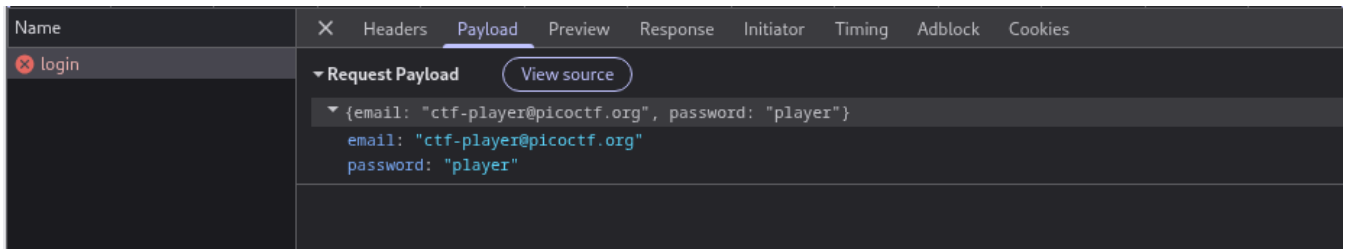
Network inspection showed POST `/login` returning 401 Unauthorized. The backend also exposed X-Powered-By: Express.

What this told me

The request reached the server successfully, but the server rejected the credentials. The status code **401 Unauthorized** confirmed that this was an authentication failure, not a broken page or missing endpoint. The **Express** header suggested a Node.js/Express backend.

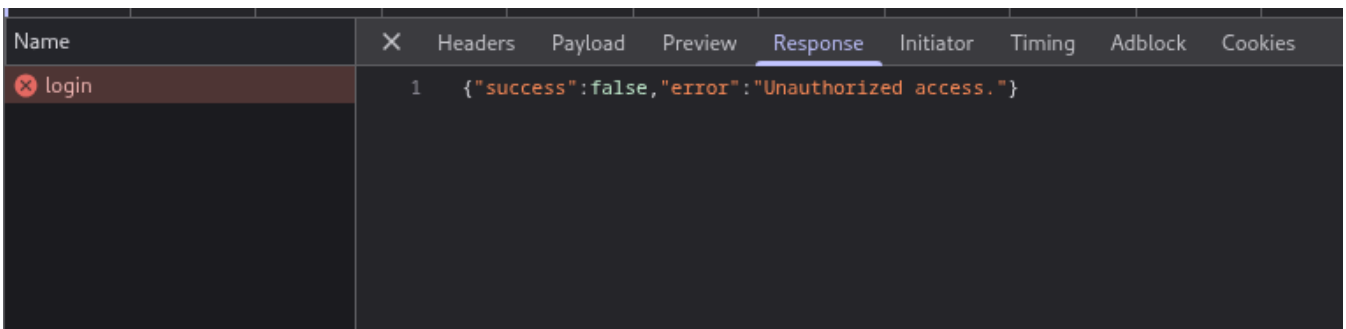
Next I checked the exact JSON payload and response so I could understand what the normal frontend was sending.

2. Checking the Payload and Server Response



The request payload contained only email and password.

```
{
  "email": "ctf-player@picoctf.org",
  "password": "player"
}
```



The server returned success:false and an Unauthorized access error.

There was no hidden third field in the normal request. However, this only proved what the frontend sends; it did **not** prove that the backend accepts only those two values. A server can still check extra headers or hidden parameters that the visible page never sends.

Important concept: what the frontend sends is not necessarily everything the backend understands.

3. Reading the Client-Side Source

Because the challenge said the developer left a secret way in, I downloaded the page source with curl instead of continuing to guess passwords.

```
curl -s http://amiable-citadel.picoctf.net:56428/ > index.html
cat index.html
```

Inside the HTML source I found this suspicious comment:

```
<!-- ABGR: Wnpx - grzcbenel olcnff: hfr urnqre "K-Qri-Npprff: lrf" -->
<!-- Remove before pushing to production! -->
```

The text looked encoded. The word **ABGR** is a strong ROT13 clue because ROT13 turns **ABGR** into **NOTE**. I decoded the full comment with:

```
echo 'ABGR: Wnpx - grzcbenel olcnff: hfr urnqre "K-Qri-Npprff: lrf"' | tr 'A-Za-z' 'N-ZA-Mn-za-m'
```

Decoded result:

```
NOTE: Jack - temporary bypass: use header "X-Dev-Access: yes"
```

This was the developer backdoor. The source code openly leaked the name and value of a custom HTTP header that bypassed normal authentication.

4. Using the Leaked Developer Header

I reproduced the normal POST /login request with curl, but added the leaked header **X-Dev-Access: yes**.

```
curl -s -X POST \
  http://amiable-citadel.picoctf.net:56428/login \
  -H 'Content-Type: application/json' \
  -H 'X-Dev-Access: yes' \
  -d '{"email":"ctf-player@picoctf.org","password":"player"}'
```

The server returned:

```
{
  "success": true,
  "email": "ctf-player@picoctf.org",
  "firstName": "pico",
  "lastName": "player",
  "flag": "picoCTF{brut4_f0rc4_b3a957eb}"
}
```

picoCTF{brut4_f0rc4_b3a957eb}

5. Why the Exploit Worked

The application had a hardcoded developer authentication bypass. Normal users had to provide valid credentials, but requests containing the special header **X-Dev-Access: yes** were accepted. The developer also left instructions for this bypass inside an HTML comment.

HTML comments and frontend JavaScript are not secret. Anyone can read them using View Source, DevTools, or curl. Likewise, a custom HTTP header is not trustworthy merely because the normal browser form does not send it; an attacker can create arbitrary headers manually.

Vulnerability: hardcoded developer authentication bypass + leaked bypass instructions in client-side source.

6. Final Solve Path

1	Try normal login	Invalid Credentials
2	Inspect Network	POST /login -> 401 Unauthorized
3	Inspect payload/response	Normal frontend sends email + password
4	Read HTML source	Suspicious ROT13 comment found
5	Decode ROT13	X-Dev-Access: yes
6	Replay POST with custom header	Authentication bypassed; flag returned

Main lesson: never store secret authentication bypasses in production code, and never assume client-side comments or custom headers are hidden from attackers.